

Technical Documentation

Sentiment Analysis Web Application

Name of Student	BITS ID	Contribution
K ROMA PAI	2024aa05965@wilp.bits-pilani.ac.in	100%
JITESH GUPTA	2024ab05020@wilp.bits-pilani.ac.in	100%
KOTHA AMITABH	2024ab05195@wilp.bits-pilani.ac.in	100%
JOHIT GARG	2024aa05907@wilp.bits-pilani.ac.in	100%
KARTHIK REDDY S	2024ab05330@wilp.bits-pilani.ac.in	100%

1. Brief Overview of the Application

This project is a web-based Sentiment Analysis application built using **Flask** and a machine learning **Voting Ensemble** classifier.

The application allows users to input text through a web interface and receive:

- Sentiment prediction (Positive / Negative/ Neutral)
- Confidence score (probabilistic output)

The machine learning model was trained using multiple classical ML algorithms combined through a Soft Voting mechanism. The final trained model and TF-IDF vectorizer are stored locally and loaded during runtime for prediction.

The project demonstrates:

- Natural Language Processing (NLP) pre-processing
- Feature engineering using TF-IDF
- Ensemble learning techniques
- Deployment of ML models using Flask

2. Instructions for Running Locally

Step 1: Clone or Download the Project

Download the project folder and navigate into it:

```
cd S1-25_AIMLCZG519_Group_65_Assignment_2_Code
```

Step 2: Create Virtual Environment (Recommended)

```
python -m venv venv  
venv\Scripts\activate
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Step 4: Run the Flask Application

```
python app.py
```

Open browser and visit:

```
http://127.0.0.1:5000/
```

3. Design Choices

3.1 Backend Framework – Flask

Flask delivers an optimal balance of simplicity and functionality for machine learning deployment. Its minimal dependencies accelerate development while efficiently routing HTTP requests, processing inputs through the prediction pipeline, and rendering results via Jinja2 templates.

Flask was chosen because:

- Lightweight and easy to configure
- Minimal overhead
- Ideal for small ML deployment projects
- Easy routing and template rendering
- Simple integration with Python ML models

Flask handles:

- HTTP request routing
- Rendering HTML templates
- Receiving user input
- Passing input to ML model
- Displaying prediction results

3.2 Machine Learning Model Training

The end-to-end training workflow transforms raw text into robust predictive features through linguistic normalization and discriminative vectorization, establishing the foundation for accurate ensemble classification across diverse input patterns.

Text Preprocessing

The preprocessing pipeline includes:

- Tokenization
- Stopword removal
- Lemmatization
- Stemming

Feature Engineering

TF-IDF vectorization strategically weights terms by inverse document frequency, amplifying discriminative sentiment words while suppressing common noise terms to create compact, informative numerical representations ideal for linear classifiers. TF-IDF Vectorization was used to convert text into numerical features.

Reason:

- Better than Bag of Words for capturing word importance
- Reduces impact of frequently occurring non-informative words

3.3 Model Selection & Ensemble Learning

Soft voting harmonizes complementary strengths across diverse classifiers, generating calibrated probability distributions that enhance prediction stability and provide transparent confidence metrics essential for user trust and decision support. Multiple models were trained:

- Logistic Regression
- Support Vector Machine
- Naive Bayes
- SGD Classifier
- Decision Tree
- Random Forest

A Soft Voting Classifier was implemented to:

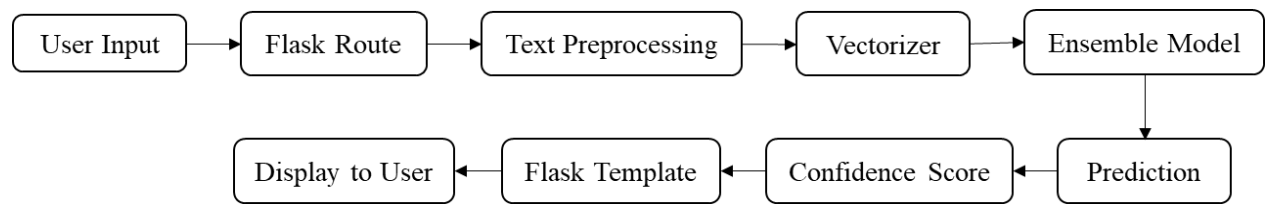
- Combine strengths of multiple models
- Improve stability
- Generate probabilistic predictions
- Provide confidence scores for UI display

Soft voting was chosen instead of hard voting because:

- It averages prediction probabilities
- Provides more reliable decision boundaries
- Enables confidence score calculation

3.4 Flask Application and Model Integration

Architecture Flow:



Integration Steps:

1. Trained model and vectorizer were saved as `.pkl` files.
2. `model.py` loads these files at startup.
3. Input text from user is preprocessed.
4. Text is transformed using stored TF-IDF vectorizer.
5. Ensemble model predicts class and probability.
6. Result is returned to Flask.
7. Flask renders prediction on the HTML page.

This separation ensures:

- Clean modular structure
- Reusable prediction logic
- Maintainable codebase

3.5 Flask and Web Application Integration

Jinja2 templating dynamically injects prediction results and confidence metrics into a clean, responsive interface designed for immediate interpretability and seamless user interaction without complex JavaScript dependencies. Flask uses Jinja2 templating. The UI was designed to:

- Accept manual text input
- Display predicted sentiment
- Display confidence score
- Maintain clean layout

4. Implementation Challenges

4.1 Model Training Challenges

Iterative refinement addressed fundamental mismatches between algorithmic strengths and sparse text representation characteristics, prioritizing linear models that excel with high-dimensional feature spaces.

1. Low Accuracy of Individual Models

Initial models such as:

- Decision Tree
- Random Forest
- Gradient Boosting

Performed poorly (~50–60% accuracy).

This was due to:

- Sparse high-dimensional TF-IDF features
- Tree-based models not handling sparse vectors efficiently
- Overfitting on small patterns

2. High Training Time

Bagging and boosting models took significantly longer to train because:

- Random Forest builds multiple trees
- Gradient Boosting builds trees sequentially
- High dimensional feature space increased computation

This caused:

- Long training time in Google Colab
- Increased memory consumption

3. Model Size Explosion

The initial ensemble model became very large due to:

- Multiple tree-based models

- Large TF-IDF vocabulary
- Storing multiple estimators

Final model size after optimizations reached ~6MB.

To manage size:

- Reduced tree depth
- Adjusted number of estimators
- Reduced TF-IDF features (if applicable)
- Prioritized linear models (Logistic Regression, SGD, SVM)

Linear models performed better on sparse text data and were more memory efficient.

4.3 UI Integration Challenges

Frontend implementation required precise coordination between model outputs and interface presentation to ensure accurate, responsive user feedback without compromising application performance.

4.3.1. Confidence Display

- Correct probability extraction
- Formatting percentage display
- Handling class index mapping properly

4.3.2. Loading Time

Ensemble models increase prediction latency.

Solutions:

- Load model once at startup
- Avoid reloading inside prediction function
- Keep preprocessing lightweight

4.3.3. NLTK Resource Handling

NLTK requires dataset downloads.

Problem: Server without internet fails

Solution:

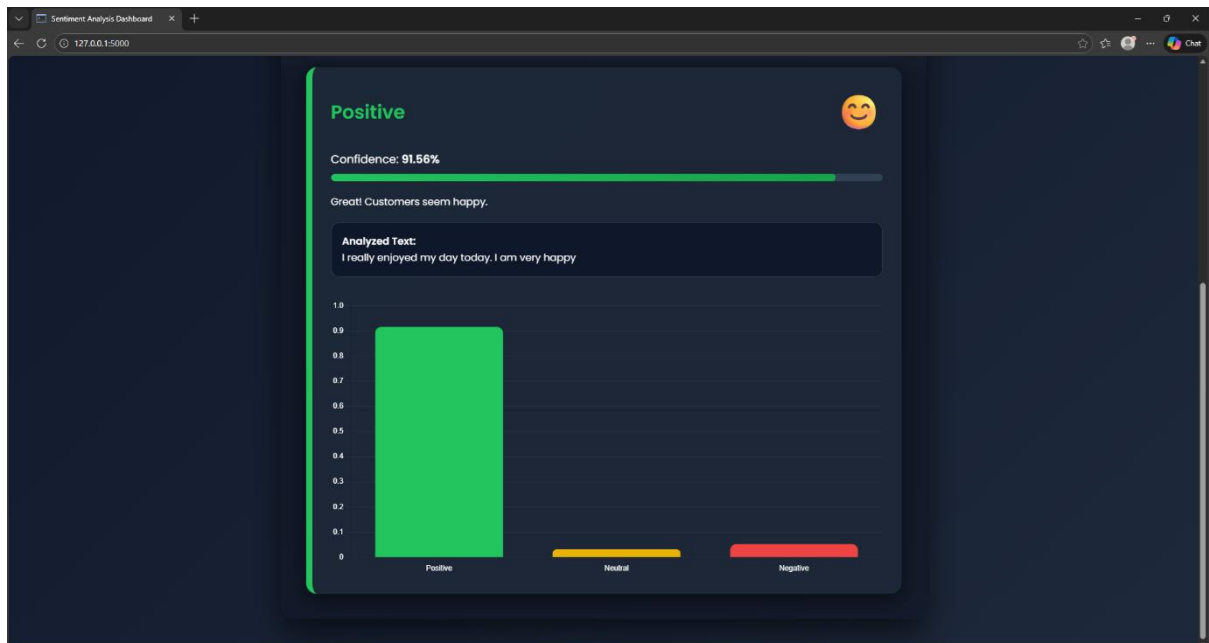
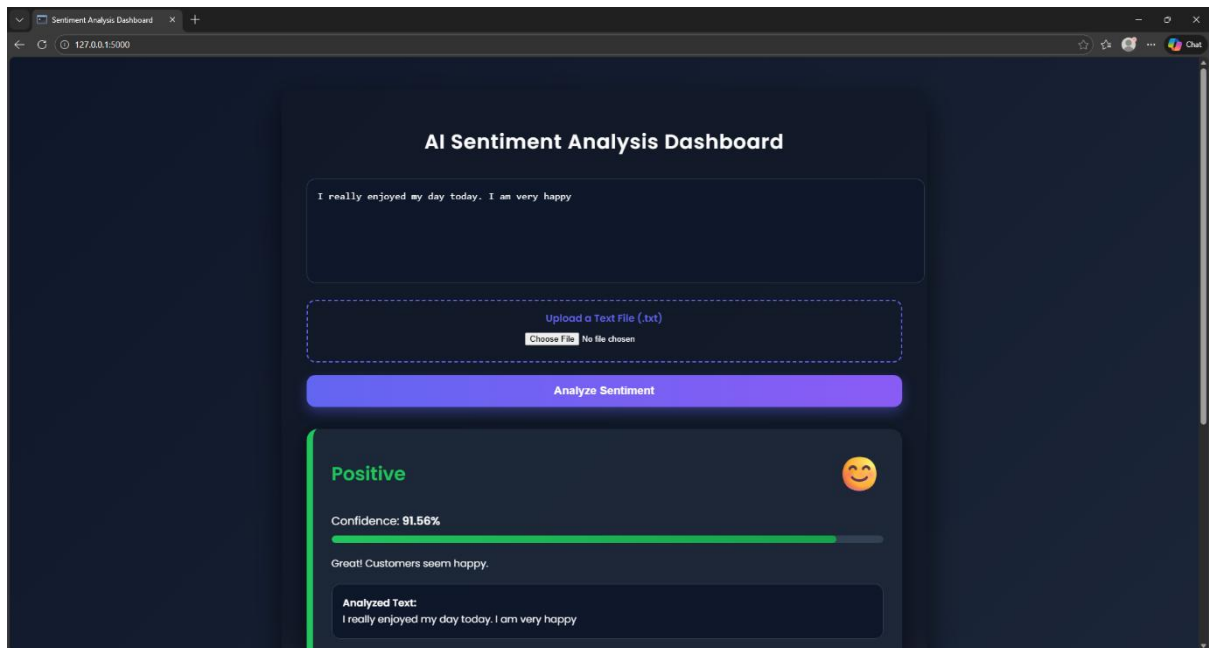
- Add runtime download
- Or manually download resources

4.4 Deployment Challenges

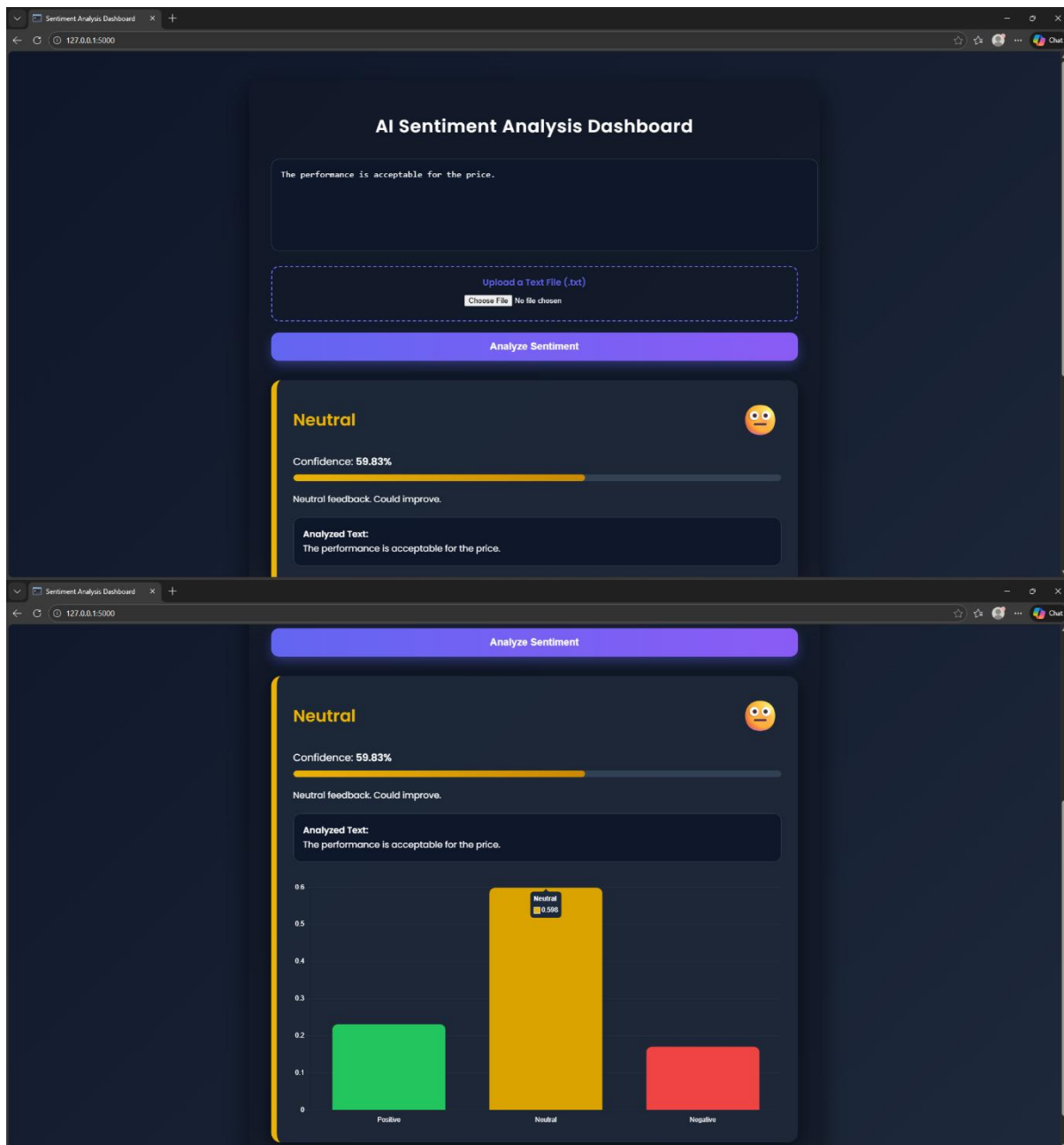
- Model size constraints
- Memory usage
- Dependency management
- Ensuring correct version of scikit-learn
- Handling file path issues between Windows and deployment environment

5. Screenshots

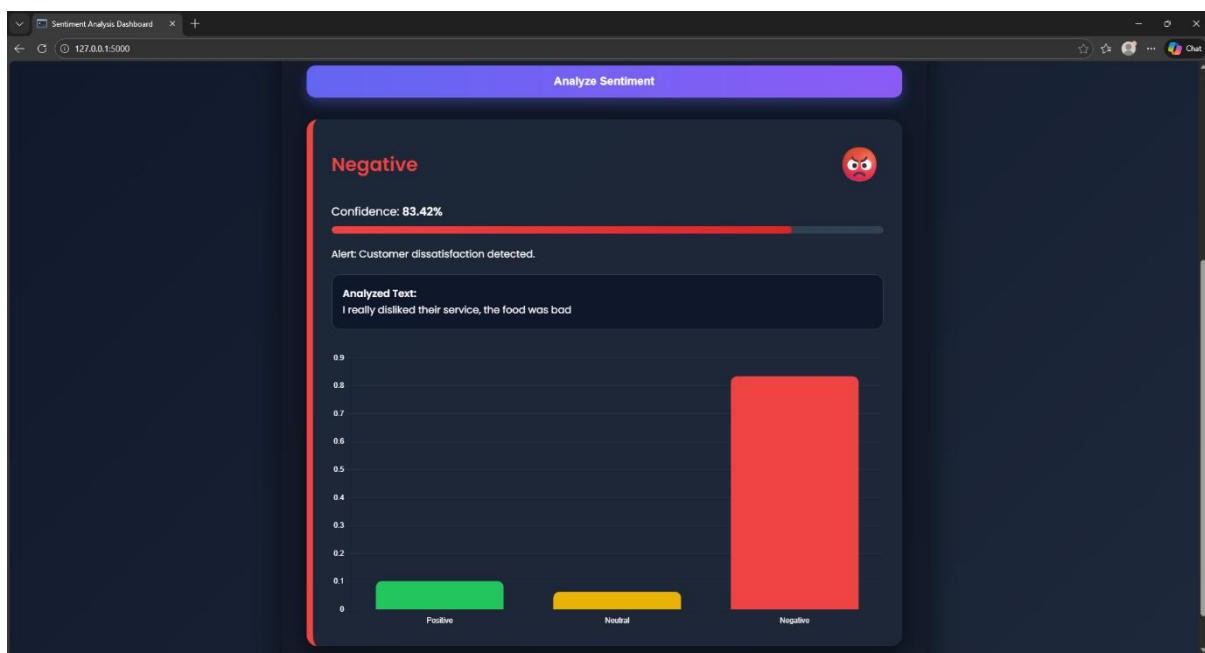
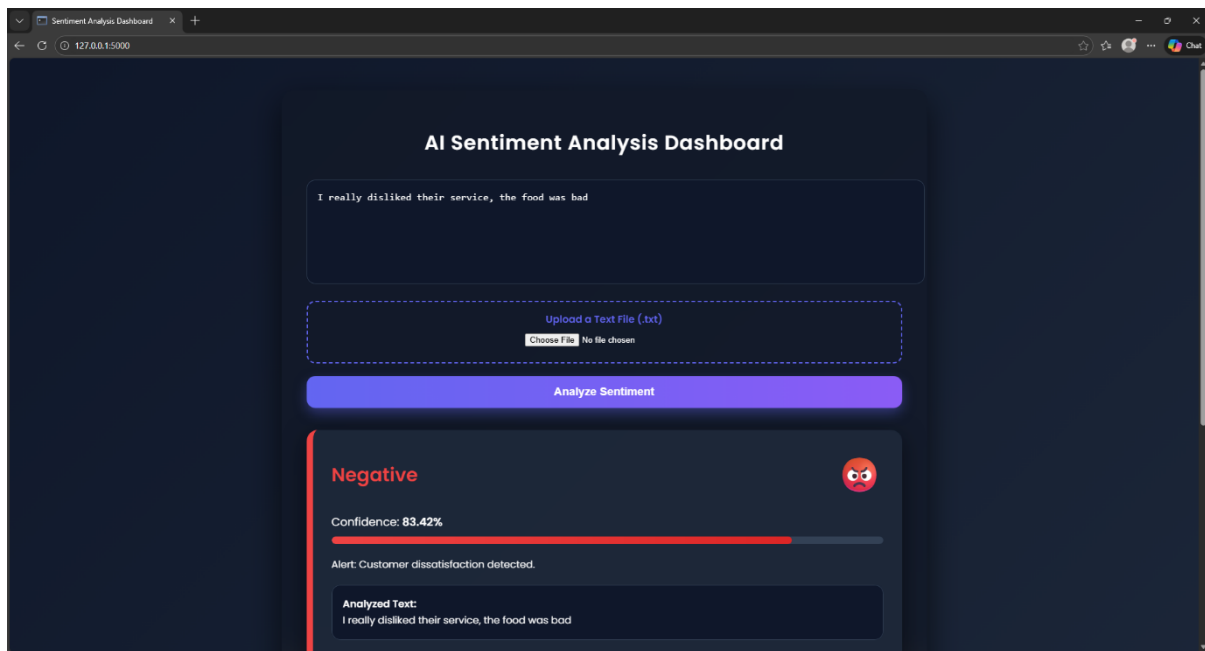
5.1 Positive Sentiment



5.2 Neutral Sentiment



5.3 Negative Sentiment



6. Conclusion

This project delivers a production-ready sentiment classifier achieving practical accuracy within strict resource constraints. The solution demonstrates thoughtful engineering trade-offs between model complexity, deployment feasibility, and user experience.

This project successfully demonstrates:

- NLP preprocessing pipeline
- Ensemble learning
- Performance trade-offs between models
- Model optimization for size and speed
- Integration of ML model with Flask web application
- End-to-end deployment of a sentiment classifier

The final application balances:

- Accuracy (~70%)
- Model size (~6MB)
- Prediction confidence
- Usability
- Modular architecture

This solution showcases practical application of NLP and machine learning deployment concepts.